# DOCUMENTAÇÃO DO SISTEMA OFICINAPRO

**Tipo de documento:** Documentação técnica de software

**Nome do projeto:** OficinaPro

**Responsável pelo desenvolvimento:** Lucas Goerler Colvero

**Data:** 27/10/2026

---

## 1. Introdução

O sistema OficinaPro é um sistema desenvolvido com o objetivo de ser um gerenciador de oficinas mecânicas, onde o mecânico ou quem gerencia pode organizar o serviço separado em clientes, veículos e ordens de serviço. Ele ajuda no gerenciamento e na organização do serviço prestado nas oficinas mecânicas, principalmente aquelas que são empresas de pequeno porte que não possuem um sistema próprio.

## 2. Objetivos do projeto

### 2.1 Objetivo geral

O objetivo geral desse sistema é ajudar a gerenciar com organização e qualidade as oficinas mecânicas nas quais não possuem um sistema de gerenciamento.

### 2.2 Objetivos específicos

Os objetivos específicos desse sistema são:

- Organizar os cadastros de clientes.
- Controlar os veículos associados aos clientes.
- Registrar e acompanhar ordens de serviço.
- Facilitar a consulta de informações.
- Associar ordens de serviços com o respectivo cliente

## 3. Escopo do sistema

O sistema contém as seguintes implementações contempladas e futuras:

### Funcionalidades contempladas

- Gerenciamento de clientes.
- Gerenciamento de veículos.
- Gerenciamento de ordens de serviço.

### Funcionalidades futuras ou não implementadas

- Gerenciamento dos mecânicos
- Gerenciamento de peças automotivas
- Gerenciamento de entrada e saída de veículos
- Compatibilidade com IOS

# 4. Requisitos do sistema

## 4.1 Requisitos funcionais

O sistema deve oferecer as seguintes funcionalidades:

| **Código** | **Descrição** |
|---|---|
| RF01 | Permitir cadastrar clientes. |
| RF02 | Permitir consultar e pesquisar clientes. |
| RF03 | Permitir editar e excluir clientes. |
| RF04 | Permitir cadastrar veículos vinculados a clientes. |
| RF05 | Permitir editar, consultar e excluir veículos. |
| RF06 | Permitir abrir ordens de serviço. |
| RF07 | Permitir acompanhar e alterar o status das ordens de serviço. |
| RF08 | Permitir editar, consultar e excluir ordens de serviço. |

## 4.2 Requisitos não funcionais

| Código | Categoria | Requisito não funcional |
|---|---|---|
| RN0F1 | Usabilidade | O sistema deve possuir uma interface intuitiva e fácil de entender |
| RN0F2 | Desempenho | As telas e listas de registros devem carregar em tempo adequado, sem atrasos |
| RN0F3 | Compatibilidade | O sistema deve ser compatível com dispositivos Android e navegadores |
| RN0F4 | Segurança | O sistema deve proteger os dados dos clientes por meio de autenticação e autorização |
| RN0F5 | Confiabilidade | O sistema deve apresentar mensagens de erros quando houver uma falha |
| RN0F6 | Manutenibilidade | O código-fonte deve ser organizado em componentes e módulos, organizado |
| RN0F7 | Integridade dos dados | O sistema deve preservar a consistência dos cadastros e dos relacionamentos entre clientes, veículos e ordens de serviço |
| RN0F8 | Acessibilidade | A interface deve apresentar textos legíveis, contraste adequado e elementos de interação fácil |
| RN0F9 | Disponibilidade | O sistema deve estar disponível para uso sempre que o servidor estiver operando |
| RN0F10 | Recuperação dos dados | O sistema deverá possuir mecanismos de backup e recuperação para evitar perdas de informações |
| RN0F11 | Segurança | O acesso ao banco de dados deve ser realizado exclusivamente pelo backend |
| RN0F12 | Desempenho | O sistema deve ter um acesso rápido e com pouco tempo de resposta |

# 5. Tecnologias utilizadas

Frontend- Interface do Sistema

| Tecnologia | Finalidade |
|---|---|
| React Native | Criação das interfaces e visual do aplicativo |
| React | Biblioteca utilizada para construir interfaces por meio dos componentes |
| TypeScript | Linguagem utilizada no desenvolvimento porque adiciona tipagem estática ao JavaScript |
| Expo | Plataforma utilizada para facilitar o desenvolvimento e a execução do aplicativo |
| NativeWind | Biblioteca utilizada para estilizar os componentes utilizando classes semelhantes às do Tailwind CSS |
| Tailwind CSS | Sistema de classes utilitárias utilizado na estilização |

Backend - Processamento e comunicação

| Tecnologia | Finalidade |
|---|---|
| Node.js | Ambiente de execução JavaScript utilizado no servidor |
| Express.js | Framework utilizado para desenvolver o servidor e criar as rotas da API |
| CORS | Utilizado para permitir requisições de diferentes origens |
| API REST | Modelo de comunicação entre o frontend e o backend por meio de requisições HTTP |

# 6. Arquitetura do sistema

A comunicação entre o Backend e Frontend acontece por meio da API REST, utilizando o formato JSON para enviar e receber informações. Quando o usuário faz uma operação, o frontend envia os dados preenchidos para o backend por meio de uma requisição HTTP.

O Servidor recebe e as executa e retorna uma resposta ao aplicativo. o Frontend interpreta a resposta e apresenta ao usuário uma mensagem indicando o resultado da operação.

Na implementação atual, o backend utiliza estruturas de dados em memórias(arrays) para armazenar temporariamente as informações dos clientes, veículos e ordens de serviço pois ainda não existe uma integração com o banco de dados.

# 7. Documentação da API

Durante o desenvolvimento, o frontend utiliza o seguinte endereço para acessar o backend: http://10.0.2.2:3000. O endereço é utilizado quando a aplicação está sendo executada em um emulador Android.

Métodos HTTP utilizados:

GET - Consultar informações
POST- Cadastrar novos registros
PUT - Atualizar registros existentes
DELETE - Excluir registros.

A API possui verificações para identificar situações em que determinado registro não é encontrado ou quando existe um problema na associação de um veículo a um cliente. As respostas de erro são enviadas em formato JSON, contendo uma mensagem explicativa.